

# Deep Learning for NLP

Student name: *Georgios Xydias*

ID: 7115152400007

---

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*

Semester: *Spring Semester 2025-2026*

---

## Prompting LLMs for Response Clarity Classification

### Contents

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>1</b> | <b>Abstract</b>                                      | <b>3</b>  |
| <b>2</b> | <b>Data</b>                                          | <b>3</b>  |
| 2.1      | Dataset Description . . . . .                        | 3         |
| 2.2      | Exploratory Data Analysis . . . . .                  | 3         |
| 2.3      | Text Preprocessing . . . . .                         | 4         |
| 2.4      | Data Partitioning . . . . .                          | 5         |
| <b>3</b> | <b>Models</b>                                        | <b>5</b>  |
| <b>4</b> | <b>Prompt Design</b>                                 | <b>5</b>  |
| <b>5</b> | <b>Prompting Strategies</b>                          | <b>6</b>  |
| 5.1      | Zero-Shot . . . . .                                  | 6         |
| 5.2      | Few-Shot . . . . .                                   | 6         |
| 5.3      | Chain-of-Thought . . . . .                           | 7         |
| 5.4      | Few-Shot Chain-of-Thought . . . . .                  | 7         |
| 5.5      | Generated Knowledge . . . . .                        | 7         |
| 5.6      | Prompt Chaining . . . . .                            | 8         |
| 5.7      | Self-Consistency . . . . .                           | 8         |
| <b>6</b> | <b>Inference Pipeline</b>                            | <b>8</b>  |
| 6.1      | Pipeline Flow . . . . .                              | 8         |
| 6.2      | Generation Parameters . . . . .                      | 9         |
| 6.3      | Output Parsing . . . . .                             | 10        |
| 6.4      | Batched Inference . . . . .                          | 10        |
| <b>7</b> | <b>Experiments and Results</b>                       | <b>11</b> |
| 7.1      | Experimental Setup . . . . .                         | 11        |
| 7.2      | Cross-Technique Comparison on Qwen3.5-0.8B . . . . . | 11        |
| 7.3      | Cross-Size Comparison on Qwen3.5 . . . . .           | 12        |
| 7.4      | Cross-Family Comparison . . . . .                    | 13        |

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| 7.5       | Structure Ablations . . . . .                                      | 14        |
| 7.6       | Sampling Effect . . . . .                                          | 16        |
| 7.7       | Thinking Mode . . . . .                                            | 16        |
| 7.8       | Summary . . . . .                                                  | 17        |
| <b>8</b>  | <b>Evaluation</b>                                                  | <b>17</b> |
| 8.1       | Confusion Matrices . . . . .                                       | 17        |
| 8.2       | Per-Class Performance . . . . .                                    | 19        |
| 8.3       | Test Set Performance . . . . .                                     | 19        |
| <b>9</b>  | <b>Error Analysis</b>                                              | <b>21</b> |
| 9.1       | Hardest Class to Predict . . . . .                                 | 21        |
| 9.2       | Performance Across Data Subgroups . . . . .                        | 22        |
| 9.3       | Concrete Error Examples . . . . .                                  | 22        |
| 9.4       | Cross-Technique Failure Modes . . . . .                            | 24        |
| 9.5       | Cross-Size Failure Modes . . . . .                                 | 24        |
| 9.6       | Fundamental Sources of Error and Ideal Model . . . . .             | 25        |
| <b>10</b> | <b>Discussion</b>                                                  | <b>26</b> |
| 10.1      | Results Analysis . . . . .                                         | 26        |
| 10.2      | Comparison with Classical ML Approaches and Transformers . . . . . | 26        |
| 10.3      | Submitted Configuration . . . . .                                  | 27        |
| 10.4      | Future Work . . . . .                                              | 28        |
| <b>A</b>  | <b>AI Assistance</b>                                               | <b>29</b> |

## 1. Abstract

This report presents our approach to response clarity classification in political question-answer pairs, as part of SemEval 2026 Task 6 [12]. The dataset, introduced by [11], consists of political interviews with U.S. presidents, where each answer is labeled as *Clear Reply*, *Ambivalent*, or *Clear Non-Reply*. We approach the task through prompting techniques applied to instruction-tuned large language models, rather than gradient-based fine-tuning. The implementation uses PyTorch [7] and the Hugging Face Transformers library [16]. The core comparison centers on three model sizes from the Qwen3.5 family [8] (0.8B, 2B, and 4B parameters), with additional cross-family experiments on Gemma 3 [10], Llama 3.2 [2], and Phi-4-mini [6]. We evaluate seven prompting strategies (zero-shot and few-shot [1], chain-of-thought [15], few-shot chain-of-thought, generated knowledge [5], prompt chaining [17], and self-consistency [14]), along with prompt-structure ablations, thinking-mode effects, and sampling sensitivity. We compare prompting methods and model sizes systematically, analyze failure modes, and contrast the prompting results with our previous work, which evaluated classical machine learning and fine-tuned encoder baselines. Our approach is based on the course slides [4, 13], the prompting reference [9], and the relevant chapters of [3]. All implementation details, additional figures, and intermediate results are available in the accompanying Jupyter notebook.

## 2. Data

### 2.1. Dataset Description

We use the dataset introduced by [11] for SemEval 2026 Task 6 [12], which consists of political interviews with U.S. presidents. Each record contains the original journalist turn (`interview_question`), the politician’s response (`interview_answer`), annotation information, metadata (e.g., title, date, president, URL), and auxiliary fields such as GPT-3.5 generated summaries. Because a single journalist turn may contain multiple questions, the dataset provides a segmented question field containing exactly one question. The core supervised instances in this work are therefore (`question`, `interview_answer`) pairs. The prediction target is `clarity_label`, a three-class label: *Clear Reply*, *Ambivalent*, and *Clear Non-Reply*. The dataset is available on Hugging Face under the identifier `ailsntua/QEvasion`, and provides two splits: 3,448 training samples and 308 test samples.

### 2.2. Exploratory Data Analysis

The full exploratory data analysis of this dataset is described in our previous work; here we summarize only the aspects that directly affect the prompting pipeline.

The class distribution is highly imbalanced: *Ambivalent* accounts for 59% of the training samples, *Clear Reply* for 31%, and *Clear Non-Reply* for only 10% (Figure 1). This imbalance motivates the use of class-balanced demonstration sampling for few-shot prompting, described later in the report. The class distribution is consistent between the training and test splits.

Text length varies across classes (Figure 2): *Ambivalent* answers tend to be the longest (mean 331 words), followed by *Clear Reply* (272 words), and *Clear Non-Reply*

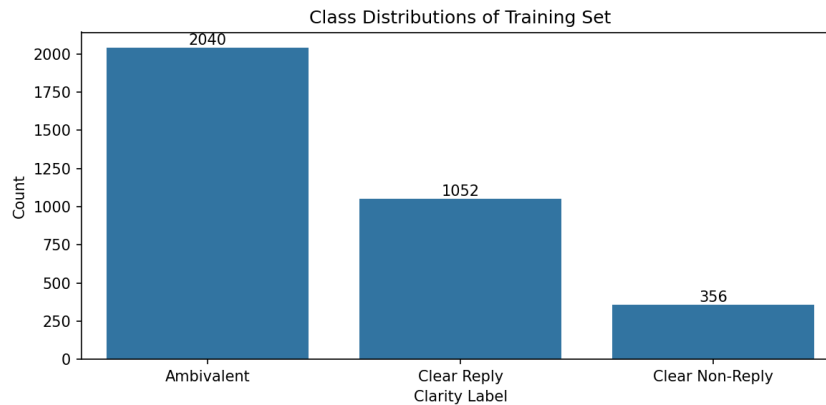


Figure 1: Class distribution in the training set.

(137 words). Unlike our previous work with fixed-length in transformer encoders, where `max_length` had to be chosen as a trade-off between coverage and truncation, no such constraint applies here: the longest combined input is approximately 2,100 words, well within the context window of every model evaluated (128K to 262K tokens). This holds even for prompting techniques that produce longer prompts, such as few-shot demonstrations or chain-of-thought.

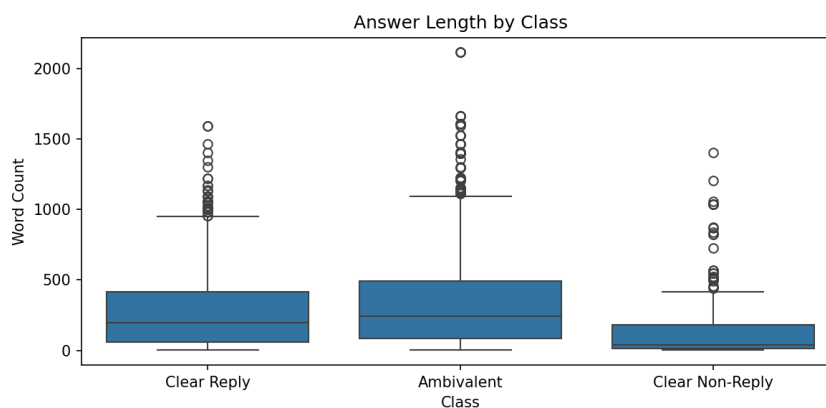


Figure 2: Answer length distribution per class.

### 2.3. Text Preprocessing

As pretrained large language models operate on raw text and rely on their own pretrained tokenizers, only one preprocessing step was applied: Deduplication. We identified and removed 58 duplicate rows (1.68%) from the training set, reducing it from 3,448 to 3,390 samples. This prevents the same example from appearing in both the few-shot demonstration pool and the held-out validation set, which would affect the validation estimates.

Unlike our previous work, which used a fixed input format applied uniformly, no global input construction is required here. Additionally, tokenization is handled internally by each model’s chat template. The way the (question, answer) pair is integrated into each prompting technique’s structure is described in a later section.

## 2.4. Data Partitioning

We reuse the 85/15 random split from our previous work on this dataset, resulting in a 2,881-sample few-shot demonstration pool and a 509-sample held-out validation set. This ratio provides a sufficiently large pool from which to retrieve few-shot demonstrations while keeping enough validation samples for reliable evaluation. Because the split is regenerated with stratification, the validation set is close to but not identical to the one used in our previous work, allowing a meaningful comparison across assignments. The test set (308 samples) is kept untouched until final prediction. A fixed random seed (SEED=42) is used throughout all experiments to ensure reproducibility.

## 3. Models

Prompting is a way of using pretrained large language models in which the task is specified entirely through the input text, with no updates to the model’s parameters [13]. The model receives a prompt that frames the problem and an input instance, and returns a textual response that is parsed into a prediction. This contrasts with fine-tuning, the approach used in our previous work, where task-specific labeled data updates the model’s weights through gradient descent. The same pretrained model can therefore handle many tasks by changing only the prompt, which is why all models in this section are accessed in their instruction-tuned variants (specifically trained to follow natural-language task instructions).

We evaluate four families of pretrained, instruction-tuned large language models, summarized in Table 1. The primary comparison includes three sizes of the Qwen3.5 family [8] (0.8B, 2B, and 4B parameters), which share a common architecture and training pipeline and thereby isolate the effect of model scale on prompting performance. To test whether the prompting patterns observed on Qwen3.5 generalize across model families, we additionally evaluate Gemma 3 [10] (1B and 4B), Llama 3.2 [2] (1B and 3B), and Phi-4-mini [6] (3.8B). All evaluated models are decoder-only transformers, accessed in their instruction-tuned variants.

| Family     | Provider        | Release        | Sizes evaluated | Context  |
|------------|-----------------|----------------|-----------------|----------|
| Qwen3.5    | Alibaba Cloud   | March 2026     | 0.8B, 2B, 4B    | 262K     |
| Gemma 3    | Google DeepMind | March 2025     | 1B, 4B          | 32K–131K |
| Llama 3.2  | Meta AI         | September 2024 | 1B, 3B          | 131K     |
| Phi-4-mini | Microsoft       | February 2025  | 3.8B            | 131K     |

Table 1: Models evaluated in this work, grouped by family.

## 4. Prompt Design

All prompts in this work follow the four-component structure introduced in the lab slides [13]. Each prompt contains an *instruction* telling the model what to do, *context* providing background and constraints, the *input* to be classified, and an *output indicator* marking where the response should begin. In our setup, these components instantiate as follows.

**Instruction.** The instruction names the task and the label space. A representative formulation is: “Classify the politician’s answer as one of three categories: Clear Reply, Ambivalent, or Clear Non-Reply.”

**Context.** The context defines the three labels and constrains the output. We include short definitions of each label drawn from the dataset’s annotation guidelines, and specify that the model must respond with exactly one of the three label strings.

**Input.** The input is the (question, answer) pair from the dataset. The question is presented first, followed by the politician’s full answer text without truncation.

**Output indicator.** The output indicator is a short marker at the end of the prompt, typically “Classification:” or “Label:”, that signals where the model should produce its prediction. In our setup, we do not write this marker into the prompt. The chat template’s `add_generation_prompt=True` setting tells the model when to start its response, and the system prompt’s “Respond with the label only” instruction tells it what to write.

Within the chat-based API of each model, the anatomy components are split across two roles. The instruction and context go into the system prompt, which sets the model’s persistent behavior as a classifier with a fixed label space and definitions. The input (question-answer pair) goes into the user prompt as the per-example content that changes between requests. The output indicator is handled implicitly by the chat template, as described above. We apply this split uniformly across all techniques and revisit the choice as an ablation in the experiments section.

## 5. Prompting Strategies

We evaluate seven prompting techniques drawn from the prompting reference [9]. All techniques share the four-component anatomy and the system-user role split from the previous section. They differ along three axes: whether labeled demonstrations are shown before the test instance, whether the model produces intermediate reasoning before its final label, and whether predictions come from a single call or from aggregating multiple calls. Each subsection below describes one technique and stating how we instantiated it for this task. Generation parameters and the output parsing logic are shared across all techniques and described in the next section.

### 5.1. Zero-Shot

Zero-shot prompting provides the model with the task instruction and the test instance, with no labeled demonstrations [1]. The model produces a prediction using only what it learned during pretraining and instruction tuning.

In our setup, we use the system prompt from [13], which holds the task description, the three label names, and the format constraint (“Respond with the label only”). The user prompt contains the test (question, answer) pair as a “Question: ... Answer: ...” formatted string. We use greedy decoding (`do_sample=False`) with `max_new_tokens=50`, since the expected output is a single label string.

### 5.2. Few-Shot

Few-shot prompting extends the zero-shot setup by including  $K$  labeled demonstra-

tions in the prompt before the test instance [1]. The demonstrations show the model labeled examples of the task at inference time, without any parameter updates.

In our setup, we use  $K = 9$  demonstrations sampled from the few-shot demonstration pool described in Section 2.4, with three demonstrations per class. We also evaluated  $K = 3$  and  $K = 6$ , as well as a uniform random sampling variant, for comparison. The class-balanced sampling addresses the imbalance reported in Section 2.2. Each demonstration is formatted as “Question: ... Answer: ... Label: ...” and placed before the test instance in the user prompt. Generation parameters match the zero-shot setup.

### 5.3. Chain-of-Thought

Chain-of-Thought (CoT) prompting asks the model to produce intermediate reasoning steps before its final answer [15]. The reasoning chain is generated as part of the model’s response, conditioning the final prediction on the model’s own intermediate analysis rather than on a direct classification.

In our setup, CoT extends the zero-shot configuration with a reasoning instruction. The system prompt is modified to ask the model to reason in steps before stating the final label, while the user prompt format remains the same. We evaluated two variants of the reasoning instruction for comparison. Because the response now includes both the reasoning chain and the label, we raise `max_new_tokens` to 300. Greedy decoding is unchanged.

### 5.4. Few-Shot Chain-of-Thought

Few-shot Chain-of-Thought (FS-CoT) combines few-shot demonstrations with chain-of-thought reasoning: each demonstration includes a reasoning chain in addition to the final label [15]. The demonstrations show the model not only what label to produce but also how to reason toward it.

In our setup, FS-CoT extends the CoT configuration with labeled demonstrations. We use  $K = 3$  demonstrations (one per class) sampled from the few-shot demonstration pool with class-balanced sampling. Each demonstration is formatted as “Question: ... Answer: ... Reasoning: ... Label: ...” and placed before the test instance in the user prompt. Because both the demonstrations and the response include reasoning chains, we raise `max_new_tokens` to 400. Greedy decoding is unchanged.

### 5.5. Generated Knowledge

Generated Knowledge prompting uses the model in two stages: first to produce relevant background knowledge about the input, then to classify the input conditioned on that knowledge [5]. The generated knowledge serves as additional context that the classification step did not have access to.

In our setup, the first stage gives the model the (question, answer) pair along with three knowledge demonstrations, and asks for relevant background information about the topic. Each demonstration shows an example of the kind of background information the model should produce for a given political response. The second stage uses a fresh prompt containing the original (question, answer) plus the generated knowledge from stage one, and asks for the final label. Each test instance therefore

requires two sequential model calls. Both stages use greedy decoding: stage one with `max_new_tokens=300` to allow a sufficiently long knowledge passage, stage two with `max_new_tokens=50`.

## 5.6. Prompt Chaining

Prompt chaining decomposes a task into multiple subtasks, with each subtask handled by a separate model call [17]. The chain breaks a complex prediction into simpler intermediate predictions that are then combined into the final output.

In our setup, the three-class classification is decomposed into three stages. Stage one asks the model whether the answer addresses the topic of the question, with a binary “Yes” or “No” response. Stage two asks the model whether the answer takes a clear position on what is being asked, also as a binary “Yes” or “No”. Stage three is a deterministic rule that maps the two binary outputs to the final three-class label: `topic = No` yields `Clear Non-Reply`; `topic = Yes` with `position = Yes` yields `Clear Reply`; `topic = Yes` with `position = No` yields `Ambivalent`. Stages one and two use greedy decoding with `max_new_tokens=10`; stage three involves no model call. Each test instance therefore requires two model calls.

## 5.7. Self-Consistency

Self-Consistency runs the same reasoning prompt multiple times with sampling and aggregates the predictions by majority vote [14]. Different sampled reasoning paths tend to converge on the same correct answer more often than they converge on any particular incorrect one, so the most frequent label across samples is typically a more reliable prediction than any single greedy run.

In our setup, Self-Consistency wraps the FS-CoT configuration. The same FS-CoT prompt is sent  $N = 3$  times with temperature  $T = 0.7$ , producing three sampled reasoning chains and three label predictions per test instance. The final label is the majority vote across the three predictions; sampled responses that fail to parse are excluded from the vote. Because running  $N = 3$  samples per instance is significantly slower than a single call, we evaluate Self-Consistency on a stratified 100-row subset of the validation set that preserves the original class distribution. Generation parameters match the FS-CoT setup (`max_new_tokens=400`), except for the switch from greedy to temperature sampling.

# 6. Inference Pipeline

This section specifies the shared inference machinery that wraps every technique: the pipeline flow that turns each test instance into a label, the generation parameters used by each technique, the output parser, and the batching and hardware setup on which all experiments were run.

## 6.1. Pipeline Flow

The canonical flow applies to the four single-call techniques (zero-shot, few-shot, CoT, FS-CoT):

```

Input:  test (question, answer)
Step 1: build_prompt(test)           -> messages
Step 2: apply_chat_template(messages) -> text
Step 3: model.generate(text)         -> raw_response
Step 4: parse(raw_response)          -> label

```

Generated Knowledge runs two sequential calls per instance, the first to produce background knowledge and the second to classify using that knowledge:

```

Input:  test (question, answer)
Step 1: build_prompt_stage1(test)     -> messages_1
Step 2: model.generate (greedy, max=300) -> knowledge
Step 3: build_prompt_stage2(test, knowledge) -> messages_2
Step 4: model.generate (greedy, max=50) -> raw
Step 5: parse(raw)                   -> label

```

Prompt Chaining runs two independent binary classification calls per instance and combines their outputs with a deterministic rule:

```

Input:  test (question, answer)
Step 1: build_prompt_topic(test)      -> messages_1
Step 2: model.generate (greedy, max=10) -> topic (Yes/No)
Step 3: build_prompt_position(test)   -> messages_2
Step 4: model.generate (greedy, max=10) -> position (Yes/No)
Step 5: combine(topic, position)      -> label

```

Self-Consistency runs the FS-CoT prompt  $N$  times with sampling and aggregates the predictions by majority vote:

```

Input:  test (question, answer)
Step 1: build_prompt_fscot(test, demos) -> messages [identical to FS-CoT]
Step 2: for i = 1..N:
        model.generate (T=0.7, max=400) -> raw_i
        label_i = parse(raw_i)
Step 3: majority_vote(label_1..N)       -> final label

```

## 6.2. Generation Parameters

Each technique uses a small set of generation parameters that control how the model produces text from the input prompt. The values differ by technique and are summarized in Table 2.

Greedy decoding (`do_sample=False`) is the default across techniques, which makes outputs deterministic given the same prompt. Self-Consistency is the only technique that uses sampling, with temperature  $T = 0.7$  to introduce variation across the three sampled runs that are then aggregated. The `max_new_tokens` budget varies with the expected response length: 50 for label-only outputs, 300 to 400 for responses that include a reasoning chain, and 10 for the binary Yes/No outputs in Prompt Chaining.

| Technique           | do_sample | Temperature | max_new_tokens |
|---------------------|-----------|-------------|----------------|
| Zero-Shot           | False     | —           | 50             |
| Few-Shot            | False     | —           | 50             |
| Chain-of-Thought    | False     | —           | 300            |
| FS-CoT              | False     | —           | 400            |
| Generated Knowledge | False     | —           | 300 / 50       |
| Prompt Chaining     | False     | —           | 10             |
| Self-Consistency    | True      | 0.7         | 400            |

Table 2: Generation parameters per technique. For Generated Knowledge, “300 / 50” denotes max\_new\_tokens for stage 1 (knowledge generation) and stage 2 (classification). For Prompt Chaining, both binary stages use max\_new\_tokens=10.

### 6.3. Output Parsing

The model produces a raw text response that needs to be reduced to a single label string for evaluation. Because techniques differ in their expected output format (label-only, reasoning chain plus label, binary Yes/No), we use four parsers, each matched to a specific output shape.

**Label-only outputs.** For zero-shot and few-shot, the model should produce a single label string. The parser strips any <think>...</think> blocks plus surrounding whitespace and punctuation, then checks whether the cleaned string matches one of the three valid labels exactly. A mismatch returns None.

**Reasoning-chain outputs.** For CoT and FS-CoT, the model produces a reasoning chain followed by the final label. The parser searches the response (case-insensitive) for occurrences of the three valid label strings and returns the one that appears latest in the text, since the model’s final label typically comes after its reasoning. A response with no valid label string anywhere returns None.

**Binary Yes/No outputs.** For Prompt Chaining stages one and two, the model should produce “Yes” or “No”. The parser strips the same noise as above, then checks the first word. If the first word is not Yes or No, it falls back to a regex search for any standalone yes or no in the response. A mismatch returns None.

**Knowledge outputs.** For Generated Knowledge stage one, the model produces background knowledge that feeds into stage two rather than a label. The parser locates a “Knowledge:” marker and extracts the text that follows, stopping at any “Input:” marker.

When the parser returns None, the prediction is left as None and counted as a miss for its true class during evaluation. We do not substitute a fallback label, since this would inflate the apparent accuracy of techniques that produce many invalid outputs.

### 6.4. Batched Inference

To speed up inference across the validation set, predictions were batched with eight prompts per model.generate call. For batched generation, the tokenizer was set to left-padding (tokenizer.padding\_side = “left”), so that every prompt in the batch ends at the same position and the model begins generating from the same offset for all sequences. The EOS token was used as the pad token.

## 7. Experiments and Results

Our experiments isolate three sources of variation in prompting performance: technique, model size, and model family. After the setup, we present the comparisons along each axis in turn, then ablations, additional studies, and the final results summary.

### 7.1. Experimental Setup

All experiments require a GPU. Kaggle’s free 30-hour GPU quota was insufficient for the full set of experiments, so we used Google Colab Pro with a single NVIDIA A100 40GB, models loaded in bfloat16 precision. The full set of experiments consumed 200 compute units in total. Even within this budget, running every prompting technique on every model size of every family is prohibitively expensive, so the experiments vary one axis at a time. All seven techniques are evaluated on Qwen3.5-0.8B (the smallest model), which serves as the cross-technique reference. The three basic techniques (zero-shot, few-shot, and Chain-of-Thought) are then carried forward to the larger Qwen3.5 sizes (2B and 4B) for the cross-size comparison and to the other families (Gemma 3, Llama 3.2, and Phi-4-mini) for the cross-family comparison.

All experiments use a fixed random seed (SEED=42) for the stratified validation split, the few-shot demonstration sampling, and the Self-Consistency subset. The validation set is the 509-row held-out partition described in Section 2.4, kept identical across all techniques and all models (except for Self-Consistency, which uses a 100-row subset). We report macro F1 (primary metric), weighted F1, and the parse failure rate for each experiment.

### 7.2. Cross-Technique Comparison on Qwen3.5-0.8B

To compare the prompting techniques themselves while holding the model fixed, we ran all seven techniques on Qwen3.5-0.8B and report the resulting macro F1, weighted F1, parse failure rate, and inference time on the validation set. Table 3 lists the variants explored per technique. Figure 3 and Table 4 summarize the final results.

| Technique           | Variants explored                                                | Final configuration             |
|---------------------|------------------------------------------------------------------|---------------------------------|
| Zero-Shot           | —                                                                | —                               |
| Few-Shot            | $K \in \{3, 6, 9\}$ , sampling $\in \{\text{random, balanced}\}$ | $K = 9$ , balanced              |
| Chain-of-Thought    | prompt variants v1, v2                                           | v2                              |
| FS-CoT              | (single configuration)                                           | $K = 3$ , balanced              |
| Generated Knowledge | (single configuration)                                           | 3 knowledge demonstrations      |
| Prompt Chaining     | (single configuration)                                           | 2 binary stages + rule          |
| Self-Consistency    | (single configuration)                                           | $N = 3$ , $T = 0.7$ , wraps FS- |

Table 3: Configurations explored per technique on Qwen3.5-0.8B. Only the final configurations are reported in Figure 3 and Table 4.

The techniques cover a wide range of macro F1 values. Few-shot with class-balanced sampling ( $K = 9$ ) achieves the highest score among single-call techniques on the full validation set. Self-Consistency reaches a higher score still, but on the smaller subset. Parse failure rates stay below 3% for all techniques except Chain-of-Thought,

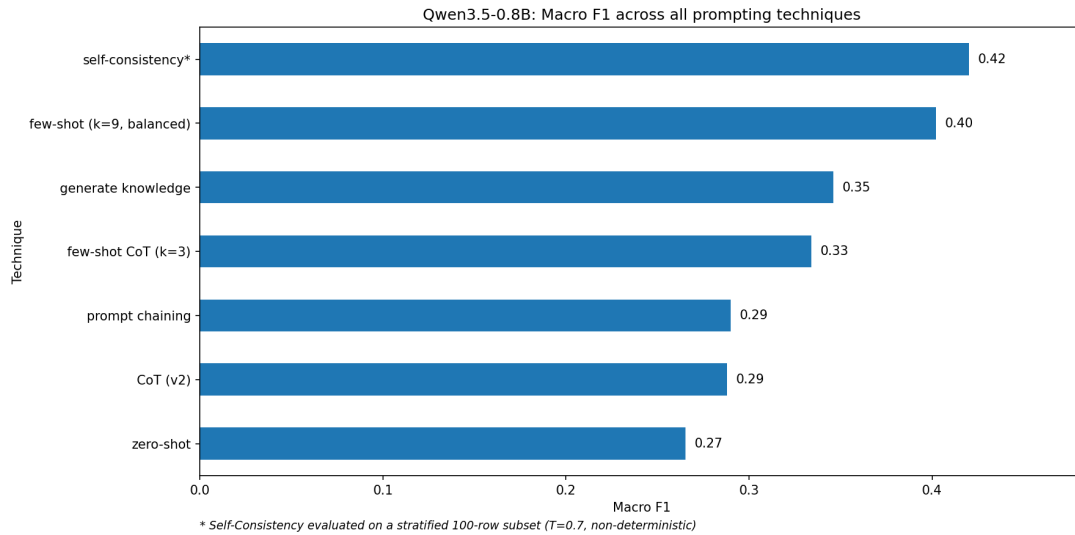


Figure 3: Macro F1 of all seven prompting techniques evaluated on Qwen3.5-0.8B on the 509-row validation set. Self-Consistency is marked with an asterisk because it was evaluated on a stratified 100-row subset under sampling ( $T = 0.7$ ), and its score is not directly comparable to the others.

| Technique                      | Macro F1 | Weighted F1 | Parse failure | Time (s) |
|--------------------------------|----------|-------------|---------------|----------|
| Zero-Shot                      | 0.265    | 0.346       | 0.4%          | 44.8     |
| Few-Shot ( $K = 9$ , balanced) | 0.402    | 0.487       | 0.2%          | 83.3     |
| Chain-of-Thought (v2)          | 0.288    | 0.253       | 11.8%         | 972.7    |
| FS-CoT ( $K = 3$ , balanced)   | 0.334    | 0.287       | 2.4%          | 1243.1   |
| Generated Knowledge            | 0.346    | 0.446       | 0.0%          | 774.4    |
| Prompt Chaining                | 0.290    | 0.240       | 0.0%          | 71.0     |
| Self-Consistency*              | 0.420    | 0.396       | 0.0%          | 798.8    |

Table 4: Detailed cross-technique results on Qwen3.5-0.8B. Parse failure rate is the fraction of responses for which the parser could not extract a valid label. Time is total inference time over the validation set, in seconds. \*Self-Consistency was evaluated on a stratified 100-row subset so its score is not directly comparable to the others.

which produces unparseable responses on roughly 12% of inputs. Inference times vary widely, with FS-CoT and CoT taking the longest.

### 7.3. Cross-Size Comparison on Qwen3.5

To examine how prompting performance scales with model size, we applied the three basic techniques (Zero-Shot, Few-Shot, and Chain-of-Thought) to the three Qwen3.5 model sizes (0.8B, 2B, and 4B). Self-Consistency, Generated Knowledge, Prompt Chaining, and FS-CoT were not included in this comparison because of the compute cost of running them on the larger sizes. Each technique was kept fixed at its final configuration from the previous subsection. Batch size was 8 for Qwen3.5-0.8B and 4 for 2B and 4B; the smaller batch size for the larger models was needed to fit them in GPU memory. Figure 4 and Table 5 summarize the results.

Zero-Shot macro F1 increases modestly with size, from 0.265 at 0.8B to 0.296 at

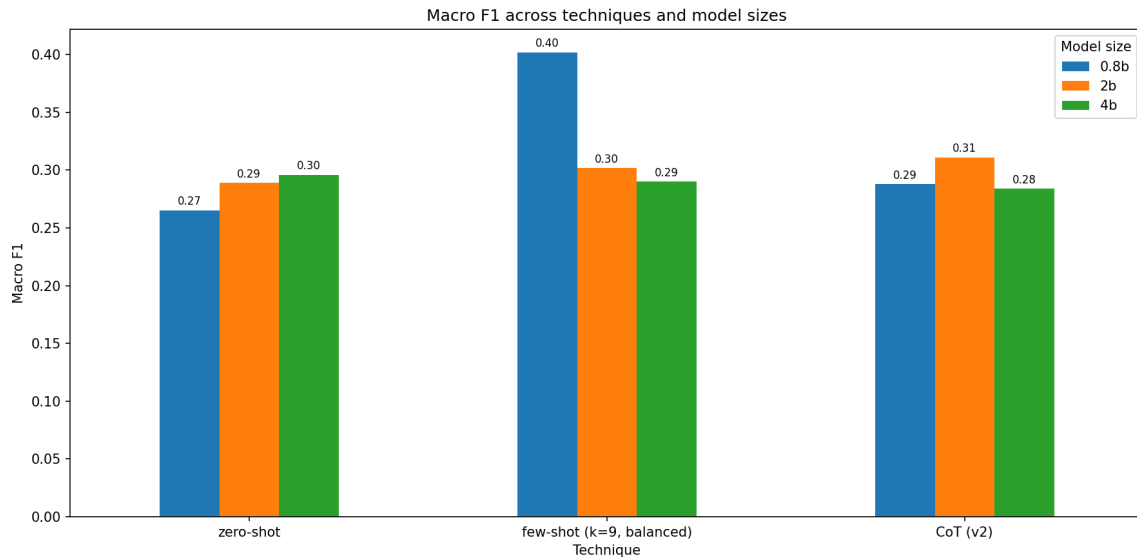


Figure 4: Macro F1 across the three basic prompting techniques on the three Qwen3.5 model sizes, evaluated on the 509-row validation set.

| Size | Technique             | Macro F1 | Weighted F1 | Parse failure | Time (s) |
|------|-----------------------|----------|-------------|---------------|----------|
| 0.8B | Zero-Shot             | 0.265    | 0.346       | 0.4%          | 44.8     |
| 0.8B | Few-Shot ( $K = 9$ )  | 0.402    | 0.487       | 0.2%          | 83.3     |
| 0.8B | Chain-of-Thought (v2) | 0.288    | 0.253       | 11.8%         | 972.7    |
| 2B   | Zero-Shot             | 0.289    | 0.194       | 0.0%          | 74.4     |
| 2B   | Few-Shot ( $K = 9$ )  | 0.302    | 0.206       | 0.0%          | 114.6    |
| 2B   | Chain-of-Thought (v2) | 0.311    | 0.315       | 31.6%         | 1946.8   |
| 4B   | Zero-Shot             | 0.296    | 0.205       | 0.0%          | 110.3    |
| 4B   | Few-Shot ( $K = 9$ )  | 0.290    | 0.207       | 0.0%          | 225.5    |
| 4B   | Chain-of-Thought (v2) | 0.284    | 0.271       | 25.5%         | 2607.8   |

Table 5: Cross-size results on Qwen3.5 for the three basic prompting techniques. Each row reports the technique’s final configuration from the previous subsection, run on a different model size.

4B. Few-Shot macro F1 decreases with size, from 0.402 at 0.8B to 0.290 at 4B, and its weighted F1 drops more sharply, from 0.487 to 0.207 (macro F1 averages per-class F1 equally while weighted F1 weights by class size, so a sharper drop in weighted F1 means the majority class is hit hardest). Chain-of-Thought macro F1 stays in the same range across all sizes, but its parse failure rate rises from 11.8% at 0.8B to 25.5% at 4B, peaking at 31.6% at 2B.

## 7.4. Cross-Family Comparison

To examine whether the patterns from the Qwen3.5 family hold for other model families, we ran the three basic techniques on five additional checkpoints: Gemma-3 (1B and 4B), Llama-3.2 (1B and 3B), and Phi-4-mini (3.8B). Each technique was kept at its final configuration from Section 7.2. A fully fair comparison would tune each prompt for each model family separately, but the compute budget kept us to a single shared configuration drawn from the Qwen3.5-0.8B results. Batch size was 8 for the 1B check-

points and 4 for the larger ones due to GPU memory. Figures 5 and 6 place these results alongside the Qwen3.5 results from earlier. Table 6 reports the results in detail.

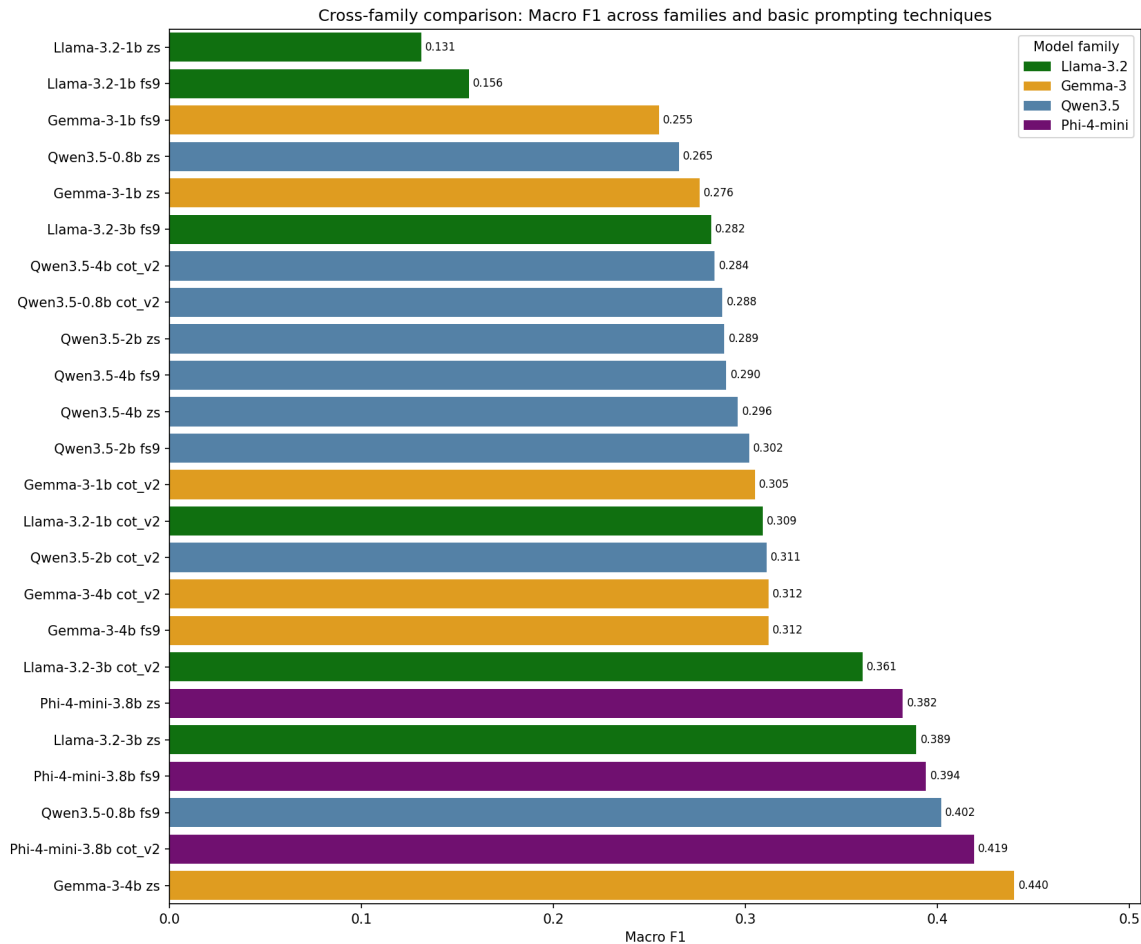


Figure 5: Macro F1 for all model and technique combinations from Sections 7.2, 7.3, and the present subsection, sorted in ascending order. Colors denote model family.

Gemma-3-4B with Zero-Shot achieves the highest macro F1 across the study, at 0.440. Phi-4-mini reaches comparable scores under all three techniques, with macro F1 between 0.382 and 0.419 and weighted F1 above macro in every row. Llama-3.2-1B Zero-Shot has a parse failure rate of 46.0%, meaning nearly half of its responses could not be assigned a valid label. Chain-of-Thought brings the largest improvement on the small Llama-3.2-1B and Gemma-3-1B checkpoints: Llama jumps from 0.131 at Zero-Shot to 0.309 at CoT-v2, and Gemma from 0.276 to 0.305. Larger checkpoints within each family generally outperform the smaller ones.

## 7.5. Structure Ablations

To test which parts of the prompt anatomy from Section 4 matter, we ran three ablations of the Zero-Shot prompt on Qwen3.5-0.8B and compared them to the baseline:

- **No labels:** drop the bullet list of the three valid labels from the system prompt.
- **No system:** leave the system message empty and move the entire system content into the user message.

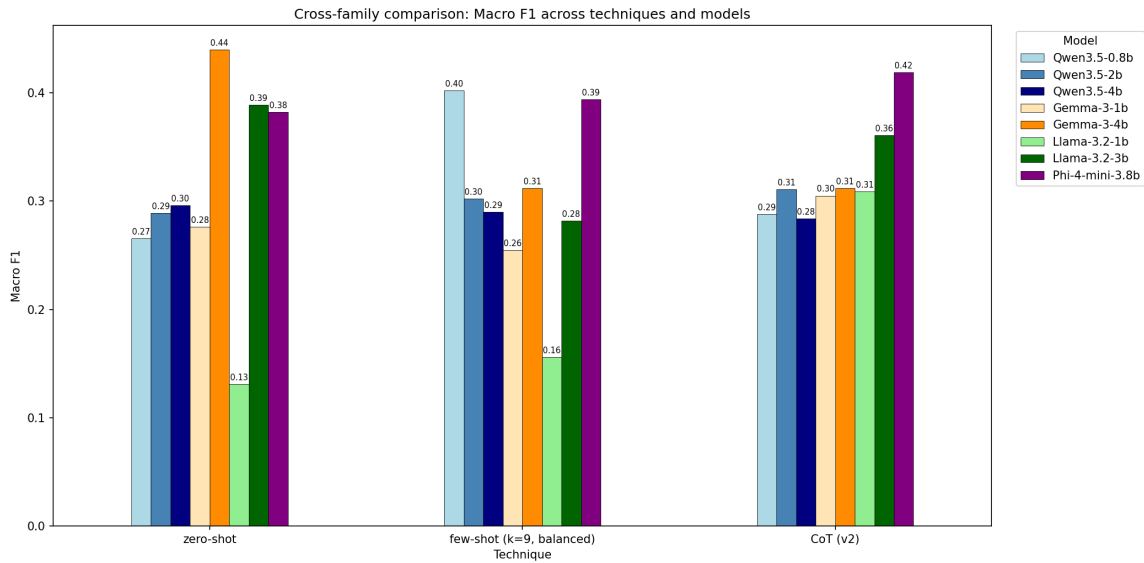


Figure 6: The same data grouped by technique. Bars within each group represent the eight model checkpoints while bar height is macro F1.

| Family     | Size | Technique             | Macro F1     | Weighted F1 | Parse failure | Time (s) |
|------------|------|-----------------------|--------------|-------------|---------------|----------|
| Gemma-3    | 1B   | Zero-Shot             | 0.276        | 0.217       | 4.1%          | 29.7     |
| Gemma-3    | 1B   | Few-Shot ( $K = 9$ )  | 0.255        | 0.360       | 1.6%          | 45.7     |
| Gemma-3    | 1B   | Chain-of-Thought (v2) | 0.305        | 0.320       | 0.4%          | 630.8    |
| Gemma-3    | 4B   | Zero-Shot             | <b>0.440</b> | 0.512       | 1.0%          | 51.8     |
| Gemma-3    | 4B   | Few-Shot ( $K = 9$ )  | 0.312        | 0.266       | 0.0%          | 116.2    |
| Gemma-3    | 4B   | Chain-of-Thought (v2) | 0.312        | 0.339       | 33.4%         | 2705.6   |
| Llama-3.2  | 1B   | Zero-Shot             | 0.131        | 0.121       | 46.0%         | 20.7     |
| Llama-3.2  | 1B   | Few-Shot ( $K = 9$ )  | 0.156        | 0.144       | 0.0%          | 33.5     |
| Llama-3.2  | 1B   | Chain-of-Thought (v2) | 0.309        | 0.357       | 0.4%          | 437.3    |
| Llama-3.2  | 3B   | Zero-Shot             | 0.389        | 0.368       | 0.0%          | 32.9     |
| Llama-3.2  | 3B   | Few-Shot ( $K = 9$ )  | 0.282        | 0.190       | 0.0%          | 82.8     |
| Llama-3.2  | 3B   | Chain-of-Thought (v2) | 0.361        | 0.349       | 3.7%          | 1331.8   |
| Phi-4-mini | 3.8B | Zero-Shot             | 0.382        | 0.535       | 0.0%          | 35.5     |
| Phi-4-mini | 3.8B | Few-Shot ( $K = 9$ )  | 0.394        | 0.526       | 0.0%          | 83.9     |
| Phi-4-mini | 3.8B | Chain-of-Thought (v2) | 0.419        | 0.513       | 0.0%          | 1001.1   |

Table 6: Cross-family results for the three basic prompting techniques. Each row reports the technique’s final configuration from Section 7.2. The best macro F1 across all model and technique pairs in the study is in bold.

- **No format:** drop the “Respond with the label only” instruction from the system prompt.

Each variant keeps everything else identical to the Zero-Shot baseline (greedy decoding, `max_new_tokens` = 50, batch size 8). Table 7 reports the results.

| Variant              | Macro F1     | Weighted F1 | Parse failure | Time (s) |
|----------------------|--------------|-------------|---------------|----------|
| Zero-Shot (baseline) | 0.265        | 0.346       | 0.4%          | 44.8     |
| No labels            | 0.000        | 0.000       | 100.0%        | 50.0     |
| No system            | <b>0.344</b> | 0.471       | 0.0%          | 42.5     |
| No format            | 0.258        | 0.415       | 20.4%         | 46.5     |

Table 7: Structure ablations on the Zero-Shot prompt with Qwen3.5-0.8B. Each variant removes one component of the prompt anatomy from Section 4. The best macro F1 among the variants is in bold.

Removing the labels list breaks the model entirely: macro F1 drops to 0.000 and parse failure rises to 100%. Removing the system role and moving all instructions into the user message gives macro F1 of 0.344, considerably above the 0.265 baseline. Despite this gain, we kept the original system-user structure from [13]. Removing the explicit format instruction leaves macro F1 nearly unchanged (0.258 versus 0.265), but parse failure rises from 0.4% to 20.4%. We did not test a complete prompt removal (empty system and no user instructions), as this would likely reproduce the No labels failure mode.

## 7.6. Sampling Effect

To test whether non-greedy decoding changes Zero-Shot performance, we ran two sampling variants of the baseline on Qwen3.5-0.8B with temperatures  $T = 0.5$  and  $T = 0.7$  (with  $\text{top-}p = 1.0$ ). Other settings match the Zero-Shot baseline (`max_new_tokens` = 50, batch size 4 for the sampling runs and 8 for greedy). Table 8 reports the results.

| Setting           | Temperature | Macro F1     | Weighted F1  | Parse failure | Time (s) |
|-------------------|-------------|--------------|--------------|---------------|----------|
| Greedy (baseline) | 0.0         | 0.265        | 0.346        | 0.4%          | 44.8     |
| Sampling          | 0.5         | <b>0.295</b> | 0.374        | 0.6%          | 76.0     |
| Sampling          | 0.7         | 0.285        | <b>0.386</b> | 0.6%          | 75.1     |

Table 8: Sampling effect on the Zero-Shot baseline with Qwen3.5-0.8B. The best score in each F1 column is in bold.

Both sampling variants improve over greedy on macro F1 and weighted F1. The best macro F1 is reached at  $T = 0.5$  (0.295 versus 0.265 at greedy), and the best weighted F1 at  $T = 0.7$  (0.386 versus 0.346). Parse failure rate stays between 0.4% and 0.6% across all three settings. Inference time increased with sampling, partly because the batch size was halved to 4 for those runs. Given the modest size of the gain, we kept greedy decoding ( $T = 0$ ) for all experiments, in line with the configuration of [13].

## 7.7. Thinking Mode

Qwen3.5 models support a built-in thinking mode [8] that generates reasoning inside `<think>...</think>` blocks before the final answer. Defaults differ by size: the 4B variant has thinking on by default, while 0.8B and 2B have it off. We re-enabled thinking explicitly via `enable_thinking=True` in `apply_chat_template` so all three Qwen3.5 sizes were tested under the same setting. The token budget was raised from 50 (the value used for the non-thinking baselines) to 512, 1024, and 2048 for the 0.8B, 2B, and 4B models respectively, so the reasoning chain had room to terminate. Decoding stayed greedy ( $T = 0$ ), so any observed effect could be attributed to thinking mode alone.

We first inspected a single validation example on each size to see how thinking behaved at each scale. On Qwen3.5-0.8B and Qwen3.5-2B, the model never picked a final label within 2048 tokens. The 0.8B got stuck repeating the same sentence, while the 2B kept producing new reasoning steps without ever deciding. Increasing the token budget further did not help. Thinking mode does not work at these sizes for this task.

On Qwen3.5-4B, we extended the test to several validation examples. The model did terminate thinking within 2048 tokens, but only on about half of them. For the inputs that finished, macro F1 was slightly above the non-thinking 4B baseline. Thinking mode works mechanically at this size, but the high failure rate and small lift make it unreliable as a default configuration.

We therefore did not include thinking mode in the cross-technique or cross-size comparisons of Sections 7.2 and 7.3.

## 7.8. Summary

The highest macro F1 observed on the validation set was 0.440, reached by Gemma-3-4B with the Zero-Shot baseline (Section 7.4). On Qwen3.5-0.8B, the best single-call result was 0.402 from Few-Shot with  $k = 9$  balanced demonstrations (Section 7.2), and Self-Consistency reached 0.420 on a 100-example subset.

Larger Qwen3.5 checkpoints did not improve over the 0.8B baseline, and Chain-of-Thought showed the highest parse-failure rate at all sizes (Section 7.3). Removing the system message raised the Zero-Shot baseline macro F1 from 0.265 to 0.344, while removing the label list dropped it to zero (Section 7.5). Sampling raised macro F1 from 0.265 to 0.295 at  $T = 0.5$  (Section 7.6). Qwen3.5’s thinking mode failed to produce parseable labels at the 0.8B and 2B sizes and was unreliable at 4B (Section 7.7).

We turn to interpretation of these findings in Section 10.

## 8. Evaluation

### 8.1. Confusion Matrices

Figures 7 and 8 show the validation-set confusion matrices for the Zero-Shot baseline and the best Few-Shot configuration (FS9-balanced) on Qwen3.5-0.8B. Both were evaluated on the same 509-sample validation set.

The two configurations exhibit different error patterns. The Zero-Shot baseline over-predicts Clear Non-Reply: 234 of 507 valid predictions (about 46%) fall in the Clear Non-Reply column, while that class accounts for only 52 of 509 samples (about 10%). Specifically, 147 Ambivalent samples and 71 Clear Reply samples are misclassified as Clear Non-Reply. The Clear Reply column receives only 71 predictions in total, against 155 true Clear Reply samples.

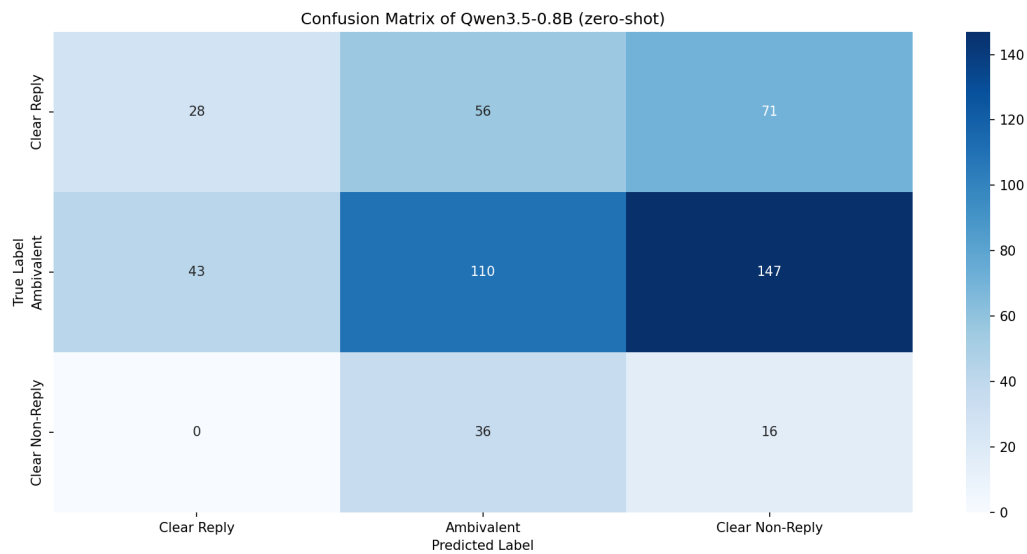


Figure 7: Confusion matrix for Qwen3.5-0.8B with Zero-Shot prompting on the validation set.

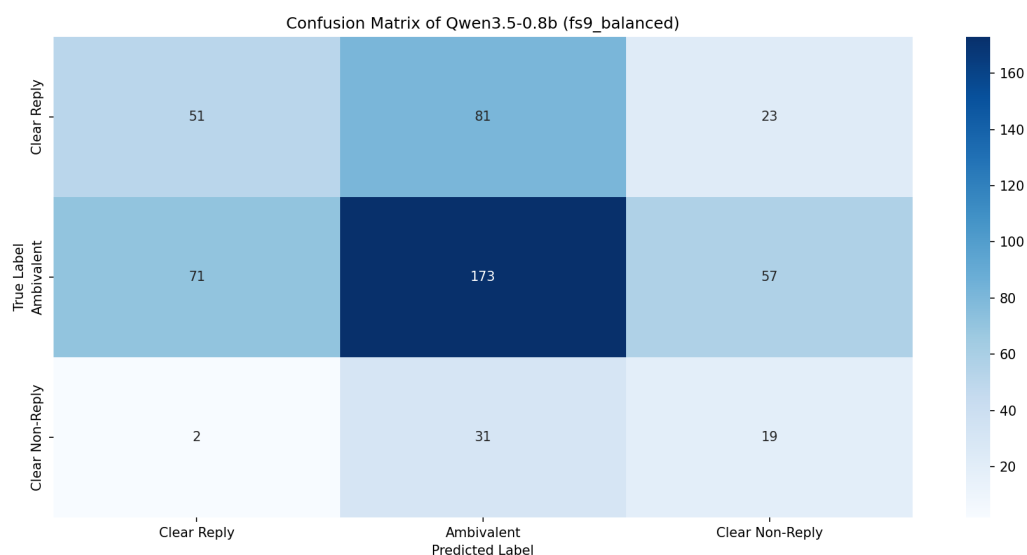


Figure 8: Confusion matrix for Qwen3.5-0.8B with FS9-balanced prompting on the validation set.

FS9-balanced shifts the prediction distribution closer to the ground-truth distribution. Ambivalent receives 285 predictions against 301 true samples, Clear Reply 124 against 155, and Clear Non-Reply 99 against 52. The dominant error is no longer the over-prediction of Clear Non-Reply but the mutual confusion between Clear Reply and Ambivalent: 81 Clear Reply samples are predicted as Ambivalent and 71 Ambivalent samples as Clear Reply, together accounting for 152 of the 265 misclassifications on this run. Clear Non-Reply remains the hardest class to recover: 31 of 52 true samples are predicted as Ambivalent and only 19 are predicted correctly. In both configurations, Clear Non-Reply is rarely confused with Clear Reply (0 instances for Zero-Shot and 2 for FS9-balanced).

## 8.2. Per-Class Performance

Table 9 reports the precision, recall, and F1 score per class for the Zero-Shot baseline and FS9-balanced (best configuration) on Qwen3.5-0.8B over the validation set.

| Configuration | Class           | Precision | Recall | F1   |
|---------------|-----------------|-----------|--------|------|
| Zero-Shot     | Clear Reply     | 0.39      | 0.18   | 0.25 |
|               | Ambivalent      | 0.54      | 0.37   | 0.44 |
|               | Clear Non-Reply | 0.07      | 0.31   | 0.11 |
|               | Macro avg       | 0.34      | 0.28   | 0.27 |
|               | Weighted avg    | 0.45      | 0.30   | 0.35 |
| FS9-balanced  | Clear Reply     | 0.41      | 0.33   | 0.36 |
|               | Ambivalent      | 0.61      | 0.57   | 0.59 |
|               | Clear Non-Reply | 0.19      | 0.37   | 0.25 |
|               | Macro avg       | 0.40      | 0.42   | 0.40 |
|               | Weighted avg    | 0.50      | 0.48   | 0.49 |

Table 9: Per-class precision, recall, and F1 for the Zero-Shot baseline and FS9-balanced on Qwen3.5-0.8B (validation set, 509 samples).

FS9-balanced improves the F1 score for every class compared to the Zero-Shot baseline: Clear Reply rises from 0.25 to 0.36, Ambivalent from 0.44 to 0.59, and Clear Non-Reply from 0.11 to 0.25. The macro-averaged F1 rises from 0.27 to 0.40 and the weighted F1 from 0.35 to 0.49.

Clear Non-Reply has the lowest F1 in both configurations (0.11 for Zero-Shot, 0.25 for FS9-balanced), with low precision (0.07 and 0.19) and recall comparable to the other classes (0.31 and 0.37). Clear Reply also has low precision (0.39 and 0.41), and its recall under Zero-Shot is the lowest of any class (0.18), rising to 0.33 with FS9-balanced. In absolute terms, Ambivalent shows the largest F1 gain from few-shot prompting (+0.15), followed by Clear Non-Reply (+0.14) and Clear Reply (+0.11).

## 8.3. Test Set Performance

We evaluated the best Qwen3.5-0.8B configuration (FS9-balanced) on the held-out test set of 308 samples, applying the same inference pipeline as on the validation set. The test set was kept completely untouched until this final evaluation. Figure 9 shows the confusion matrix and Table 10 reports the per-class metrics.

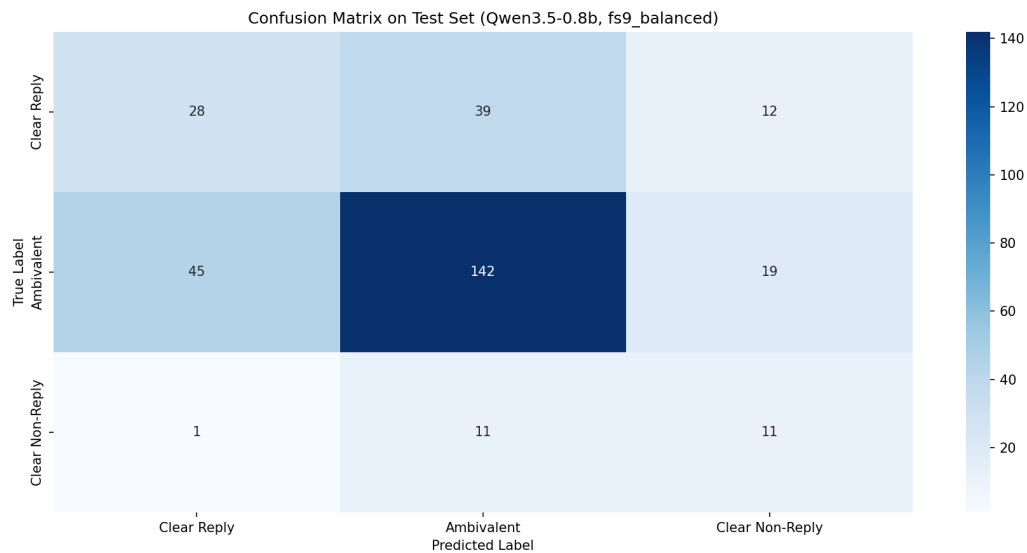


Figure 9: Confusion matrix for Qwen3.5-0.8B with FS9-balanced prompting on the test set.

| Class           | Precision | Recall | F1   | Support |
|-----------------|-----------|--------|------|---------|
| Clear Reply     | 0.38      | 0.35   | 0.37 | 79      |
| Ambivalent      | 0.74      | 0.69   | 0.71 | 206     |
| Clear Non-Reply | 0.26      | 0.48   | 0.34 | 23      |
| Macro avg       | 0.46      | 0.51   | 0.47 | 308     |
| Weighted avg    | 0.61      | 0.59   | 0.60 | 308     |

Table 10: Per-class precision, recall, and F1 for Qwen3.5-0.8B with FS9-balanced prompting on the test set.

Performance on the test set is higher than on the validation set: macro F1 rises from 0.40 to 0.47 and weighted F1 from 0.49 to 0.60. The class distribution differs across the two splits: Ambivalent accounts for 206 of 308 test samples (about 67%) versus 301 of 509 validation samples (about 59%), and Clear Non-Reply for 23 of 308 (about 7%) versus 52 of 509 (about 10%). The largest per-class F1 improvement is on Ambivalent (0.59 to 0.71), followed by Clear Non-Reply (0.25 to 0.34); Clear Reply F1 is essentially unchanged (0.36 to 0.37).

The dominant error pattern is the same as on the validation set: mutual confusion between Clear Reply and Ambivalent accounts for 84 of the 127 misclassifications (45 true Ambivalent predicted as Clear Reply, 39 true Clear Reply predicted as Ambivalent). Clear Non-Reply has the largest precision-recall asymmetry, with recall (0.48) almost twice precision (0.26). No parse failures occurred on the test set.

## 9. Error Analysis

This section focuses on the Qwen3.5 family using validation-set predictions. Subsections 9.1 through 9.3 examine the best configuration (FS9-balanced on Qwen3.5-0.8B), with the Zero-Shot baseline included. Subsections 9.4 and 9.5 extend the analysis along two axes: cross-technique on Qwen3.5-0.8B (Zero-Shot, FS9-balanced, CoT, Generated Knowledge, and Chain-of-Clarity) and cross-size on the FS9-balanced configuration (0.8B, 2B, and 4B). Section 9.6 discusses fundamental sources of error and what an ideal setup would need.

### 9.1. Hardest Class to Predict

As shown in Section 8.2, Clear Non-Reply is consistently the hardest class to predict for both configurations, with the lowest per-class F1 (0.11 for Zero-Shot and 0.25 for FS9-balanced). Table 11 shows the most common misclassification pairs for each configuration on the validation set.

| True Label      | Predicted       | Zero-Shot | FS9-balanced |
|-----------------|-----------------|-----------|--------------|
| Ambivalent      | Clear Non-Reply | 147       | 57           |
| Clear Reply     | Ambivalent      | 56        | 81           |
| Ambivalent      | Clear Reply     | 43        | 71           |
| Clear Reply     | Clear Non-Reply | 71        | 23           |
| Clear Non-Reply | Ambivalent      | 36        | 31           |
| Clear Non-Reply | Clear Reply     | 0         | 2            |

Table 11: Most common misclassification pairs for Zero-Shot and FS9-balanced on Qwen3.5-0.8B (validation set).

The dominant error pattern differs between the two configurations. Under Zero-Shot, the largest error is Ambivalent samples predicted as Clear Non-Reply (147 of 353 total errors, about 42%), followed by Clear Reply samples predicted as Clear Non-Reply (71). The Zero-Shot baseline over-predicts Clear Non-Reply: of its 234 valid Clear Non-Reply predictions, only 16 are correct. Under FS9-balanced, the dominant errors are the mutual confusion between Clear Reply and Ambivalent: Clear Reply samples predicted as Ambivalent (81) and Ambivalent samples predicted as Clear Reply (71) together account for 152 of 265 total errors (about 57%).

In both configurations, Clear Non-Reply is rarely confused with Clear Reply (0 errors for Zero-Shot, 2 for FS9-balanced), while errors involving Ambivalent (either as true or predicted) account for most of the remaining mistakes. This suggests that the boundary between direct answers and complete evasions is relatively well-defined in the model’s outputs, while the boundaries involving the middle class (Ambivalent) drive most of the confusion.

## 9.2. Performance Across Data Subgroups

Section 2.2 established that inputs are passed to inference without truncation. Any length-related performance effects would therefore arise from other mechanisms, such as the relevance of few-shot demonstrations to inputs of different lengths. We analyzed error rates across answer length and question length groups under FS9-balanced on Qwen3.5-0.8B to identify whether such effects appear in practice. Tables 12 and 13 show the results.

| Answer Length     | Samples | Accuracy |
|-------------------|---------|----------|
| Short (< 50)      | 104     | 0.442    |
| Medium (50–150)   | 98      | 0.449    |
| Long (150–300)    | 101     | 0.525    |
| Very Long (> 300) | 206     | 0.485    |

Table 12: Accuracy by answer length group under FS9-balanced on Qwen3.5-0.8B (validation set).

| Question Length  | Samples | Accuracy |
|------------------|---------|----------|
| Short (< 10)     | 183     | 0.464    |
| Medium (10–20)   | 221     | 0.507    |
| Long (20–30)     | 75      | 0.427    |
| Very Long (> 30) | 30      | 0.467    |

Table 13: Accuracy by question length group under FS9-balanced on Qwen3.5-0.8B (validation set).

For answer length, the model performs best on long answers (150–300 words, accuracy 0.525) and worst on short and medium answers (0.442 and 0.449 respectively). Very long answers (> 300 words) reach 0.485 accuracy, between the long and short-/medium groups. The largest bucket is Very Long with 206 samples (about 40% of the validation set), consistent with the long-tailed distribution of answer lengths observed in Section 2.2.

For question length, accuracy is highest on medium questions (10–20 words, accuracy 0.507) and lowest on long questions (20–30 words, accuracy 0.427). Short and very long questions land in between (0.464 and 0.467). The very long group is small (30 samples), so its accuracy estimate has wider uncertainty than the other buckets.

## 9.3. Concrete Error Examples

To better understand the nature of model failures, we examined specific misclassified examples from the validation set under FS9-balanced on Qwen3.5-0.8B. The three cases below illustrate distinct failure patterns.

**Example 1: Clear Reply predicted as Ambivalent.**

*Question:* “Could the terror threats be the reason you have skipped Kenya in your visits to Africa?”

*Answer (truncated):* “Oh, no, no, no. [...] So we have been very consistent in pushing governments not just in Africa, but around the world, to respect the right of journalists to practice their trade as a critical part of civil society. [...] Now, with respect to terrorism, I think there’s uniform concern of terrorist infiltration in many countries throughout Africa...”

The question asks whether terror threats led the politician to skip Kenya. The first three words (“Oh, no, no, no”) are a direct denial. However, the response continues with several hundred words of elaboration on good governance, media freedom, and journalism policy before returning to terrorism in Africa much later. Despite the explicit denial at the start, the answer’s length and the mostly off-topic content likely led the model to predict Ambivalent.

**Example 2: Ambivalent predicted as Clear Reply.**

*Question:* “What will you tell President Putin about NATO?”

*Answer (truncated):* “Well, I think he’s going to see about this summit – this has turned out to be a very successful summit. [...] The 2 percent was a range, a goal. It wasn’t something that they were committed to. Now it’s a commitment. [...] I think he’s going to see that there’s great unity, great spirit, great esprit de corps. And I think we’re going to have a good meeting.”

The question asks specifically what the politician will tell Putin. The answer describes NATO’s unity and the 2 percent commitment in detail but never states what will actually be communicated to Putin. The confident tone and concrete details (specific percentages, summit outcomes) make the response sound like a clear reply, which likely led the model to label it as Clear Reply despite the actual question being unanswered.

**Example 3: Clear Non-Reply predicted as Ambivalent.**

*Question:* “Do you now see Kim Jong Un as an equal?”

*Answer:* “In what way?”

The politician deflects the question with a counter-question rather than providing any substantive response. This is annotated as a clear non-reply by deflection. However, the shortness of the answer (three words) and the absence of any commitment to a position likely led the model to predict Ambivalent rather than Clear Non-Reply.

These examples reveal three recurring patterns. In Example 1, a direct answer is buried under extensive elaboration on related topics, leading the model to read the response as ambivalent despite a direct opening. In Example 2, confident tone and concrete details mask the absence of an actual answer to the question asked. In Example 3, a brief deflection through counter-question fails to register as a complete non-reply because the model appears to rely on rich content as signal for clarity. In all three cases, the errors are consistent with the model being misled by surface-level signals such as answer length, tone, and amount of content, rather than by a deeper evaluation of whether the specific question was directly addressed.

## 9.4. Cross-Technique Failure Modes

To examine whether different prompting techniques fail in qualitatively different ways, we compared per-class accuracy and dominant errors across some prompting techniques on Qwen3.5-0.8B over the validation set. Table 14 summarizes the results.

| Technique           | CR    | Amb   | CNR   | Top misclassification              | Parse fail |
|---------------------|-------|-------|-------|------------------------------------|------------|
| Zero-Shot           | 0.179 | 0.365 | 0.308 | Ambivalent → Clear Non-Reply (147) | 0.4%       |
| FS9-balanced        | 0.327 | 0.575 | 0.365 | Clear Reply → Ambivalent (81)      | 0.2%       |
| CoT                 | 0.571 | 0.083 | 0.558 | Ambivalent → Clear Reply (129)     | 11.8%      |
| Generated Knowledge | 0.429 | 0.472 | 0.154 | Ambivalent → Clear Reply (101)     | 0%         |
| Prompt Chaining     | 0.551 | 0.060 | 0.904 | Ambivalent → Clear Non-Reply (166) | 0%         |

Table 14: Per-class accuracy (CR = Clear Reply, Amb = Ambivalent, CNR = Clear Non-Reply), most common misclassification, and parse failure rate per technique on Qwen3.5-0.8B (validation set, 509 samples).

The techniques differ substantially in which classes they recognize. FS9-balanced is the only technique that maintains moderate accuracy across all three classes. Zero-Shot has the lowest accuracy on Clear Reply (0.179) and over-predicts Clear Non-Reply (147 Ambivalent samples misclassified as Clear Non-Reply). CoT and Chain-of-Clarity show the most extreme behavior: both reach the highest accuracy on Clear Non-Reply (0.558 and 0.904 respectively) but collapse on Ambivalent (0.083 and 0.060), pushing predictions toward the two extreme labels and rarely committing to the middle class. Generated Knowledge sits between FS9-balanced and Chain-of-Clarity, with moderate accuracy on Clear Reply (0.429) and Ambivalent (0.472) but the lowest accuracy on Clear Non-Reply (0.154).

Parse failure rates also vary widely. CoT shows the highest rate at 11.8%, reflecting verbose reasoning chains that do not consistently terminate with a parseable label within the token budget. The other four techniques all stay below 0.5%.

Across all techniques, Ambivalent is the dominant misclassification pair, either as true label (over-predicted toward Clear Reply or Clear Non-Reply depending on technique) or as predicted label. No technique avoids confusion involving the middle class, suggesting that the boundary between Ambivalent and the two extreme labels is the hardest to handle regardless of the prompting strategy.

## 9.5. Cross-Size Failure Modes

To examine how scaling the base model affects failure patterns, we compared per-class accuracy and dominant errors for FS9-balanced across Qwen3.5-0.8B, 2B, and 4B over the validation set. Table 15 summarizes the results.

Scaling the base model changes the failure pattern substantially. The 0.8B configuration is the only one that maintains moderate accuracy on Ambivalent (0.575). At 2B and 4B, Ambivalent accuracy collapses to 0.007 even though the same prompt and demonstrations are used. The larger models gain substantially on the two extreme classes (Clear Reply rises from 0.327 to 0.859 at 2B, Clear Non-Reply from 0.365 to 0.712), but at the cost of almost entirely abandoning the middle class.

The dominant error also reverses direction. At 0.8B, the top error is Clear Reply samples misclassified as Ambivalent (81 cases). At 2B and 4B, the top error becomes

| Size | CR    | Amb   | CNR   | Top misclassification          | Parse fail |
|------|-------|-------|-------|--------------------------------|------------|
| 0.8B | 0.327 | 0.575 | 0.365 | Clear Reply → Ambivalent (81)  | 0.2%       |
| 2B   | 0.859 | 0.007 | 0.712 | Ambivalent → Clear Reply (207) | 0%         |
| 4B   | 0.833 | 0.007 | 0.712 | Ambivalent → Clear Reply (175) | 0%         |

Table 15: Per-class accuracy (CR = Clear Reply, Amb = Ambivalent, CNR = Clear Non-Reply), most common misclassification, and parse failure rate for FS9-balanced across three Qwen3.5 sizes (validation set, 509 samples).

Ambivalent samples misclassified as Clear Reply (207 and 175 cases), with much larger absolute counts. The 2B and 4B models are not making fewer errors than the 0.8B model; they are making different ones, and more of them on the majority class.

Parse failure rates drop to zero at 2B and 4B (versus 0.2% at 0.8B), so the change in error patterns is not caused by output-format issues. These observations are consistent with the macro F1 drop reported in Section 7.3 (0.402 at 0.8B, 0.302 at 2B, 0.290 at 4B) and indicate that scaling the base model under the same prompt biases the classifier toward the two extreme labels.

## 9.6. Fundamental Sources of Error and Ideal Model

The core difficulty of this task lies in the nature of political communication itself. Politicians often give responses that sound like answers without addressing the question directly, making the boundary between Clear Reply and Ambivalent fundamentally hard to define. Our analysis identifies three factors that contribute to the errors.

First, the errors examined in Section 9.3 are consistent with the model weighing surface-level signals (answer length, tone, amount of substantive content) more heavily than the semantic alignment between question and answer. A long, confident response is often predicted as Clear Reply even when the actual question remains unanswered (Example 2), and a detailed but indirect answer is predicted as Ambivalent even when it does address the question (Example 1).

Second, the Ambivalent class is involved in the dominant misclassification of every technique tested (Section 9.4). Across Zero-Shot, FS9-balanced, CoT, Generated Knowledge, and Prompt Chaining, Ambivalent appears in the top error pair either as true label or as predicted label. This indicates that the middle class is the hardest to isolate regardless of prompting strategy and that the issue is more structural than technique-specific.

Third, scaling the base model does not stabilize the classifier. As shown in Section 9.5, larger Qwen3.5 models under the same prompt collapse on Ambivalent (accuracy 0.007 at 2B and 4B versus 0.575 at 0.8B), polarizing predictions toward the two extreme labels. Improvements from prompting are therefore not guaranteed to transfer across model sizes within the same family.

An ideal setup for this task would need to go beyond surface-level pattern matching and develop a form of pragmatic understanding: the ability to determine whether the core intent of the question was addressed, regardless of how the answer is framed. Approaches that may help include fine-tuning on labeled question-answer pairs to learn the class definitions, and richer multi-step prompting setups than those evaluated here. For instance, the Generated Knowledge variant in this work used three demonstrations, while the original formulation [5] uses around 20; scaling the demon-

stration pool, combining generated knowledge with chain-of-thought reasoning (especially when the reasoning demonstrations are well-defined by domain specialists) may yield further improvements.

## 10. Discussion

### 10.1. Results Analysis

Among the techniques evaluated on Qwen3.5-0.8B, FS9-balanced achieved the best single-call validation macro F1 (0.402), with Self-Consistency reaching 0.420 on a 100-sample subset and Zero-Shot scoring 0.265. The validation set was used throughout to compare techniques, model sizes, and prompt designs; the test set was reserved for the final evaluation of the best-performing configuration. FS9-balanced on Qwen3.5-0.8B reached macro F1 0.47 and weighted F1 0.60 on the test set.

Two findings went against the expectation that larger models or richer prompts would help. First, scaling Qwen3.5 under the same FS9-balanced prompt did not improve macro F1: from 0.402 at 0.8B to 0.302 at 2B and 0.290 at 4B (Section 7.3). The per-class analysis in Section 9.5 shows that the 2B and 4B models almost entirely stop predicting Ambivalent (accuracy 0.007), pushing predictions toward the two extreme labels. Second, removing the system message from the Zero-Shot baseline lifted macro F1 from 0.265 to 0.344 (Section 7.5), suggesting that prompt structure designed for larger instruction-tuned models may be too restrictive for small open-weight base models.

Comparing across model families also shows that family matters in this range. Gemma-3-4B with the Zero-Shot baseline reached the best validation macro F1 in this study (0.440), higher than any Qwen3.5 result under any technique, though at about five times the parameter count of the Qwen3.5-0.8B used as our main configuration. At equal parameter count, Gemma-3-4B also outperformed Qwen3.5-4B’s FS9-balanced result of 0.290, suggesting that the family-level difference is not only a size effect. Phi-4-mini-3.8B stayed consistent across techniques (weighted F1 above macro F1 in every row, without the class collapse seen in larger Qwen3.5 models), while Llama-3.2-1B failed on Zero-Shot with a 46% parse failure rate. These differences likely come from the different training and alignment recipes used by each model family.

Several observed differences are small. The Self-Consistency macro F1 (0.420 on a 100-sample subset) is slightly higher than FS9-balanced’s 0.402 on the full validation set and the temperature-sampling gain over greedy decoding (0.295 vs 0.265 at  $T = 0.5$ ) is similarly small.

### 10.2. Comparison with Classical ML Approaches and Transformers

In our earlier work on this dataset, we applied classical machine learning approaches (TF-IDF and GloVe word embeddings with Logistic Regression) and fine-tuned transformer encoders (BERT, DistilBERT, DeBERTa). Table 16 compares the best results from each approach with the best prompting result reported in this work on the same test set of 308 samples.

On the test set, the three approaches produce a clear ordering: fine-tuned transformers (0.55–0.61) outperform prompting (0.47), which in turn outperforms classical machine learning (0.41–0.43).

| Approach                              | Test Macro F1 |
|---------------------------------------|---------------|
| TF-IDF + LR (classical ML)            | 0.41          |
| GloVe + LR (classical ML)             | 0.43          |
| BERT (fine-tuned transformer)         | 0.61          |
| DistilBERT (fine-tuned transformer)   | 0.55          |
| DeBERTa (fine-tuned transformer)      | 0.57          |
| Qwen3.5-0.8B FS9-balanced (prompting) | 0.47          |

Table 16: Comparison of three approaches on the same test set (308 samples).

The advantage of prompting over classical machine learning likely comes from the richer pretrained representations of large language models compared to TF-IDF or static word embeddings, together with the general reasoning capability that lets them recognize indirect responses without seeing many task-specific examples. Even with only nine balanced demonstrations, FS9-balanced on Qwen3.5-0.8B reaches a test macro F1 four points above the best classical baseline.

Fine-tuned transformers outperform prompting because they can update their parameters using the full 2,881 training examples, learning task-specific patterns for the three clarity classes. Prompting changes the input but not the parameters, so the model’s internal view of what distinguishes Clear Reply from Ambivalent remains the same across techniques. The qualitative failure mode, however, is similar in both approaches: in our earlier work, fine-tuned encoders showed Clear Reply and Ambivalent confusion as the dominant error, the same pattern observed here under FS9-balanced (Section 9.1). What changes is the frequency. Fine-tuning reduces these errors through gradient updates on labeled examples but does not eliminate them, since the boundary between partial and full responses is inherently fuzzy.

In terms of practical trade-offs, prompting requires no training time and no gradient computation, making it faster to deploy than fine-tuning. Classical machine learning trains in seconds without GPU resources, but its lower test accuracy makes prompting a stronger option when GPU access is available. Fine-tuned transformers offer the best test performance but require labeled training data, model-specific hyperparameter tuning, and longer training times.

### 10.3. Submitted Configuration

The submitted configuration was FS9-balanced on Qwen3.5-0.8B, the best-performing single-call setup on the validation set. The notebook was re-executed on Kaggle’s NVIDIA T4 environment (16 GB memory per device) to produce the final test-set predictions.

For the re-execution, only the FS9-balanced inference pipeline and its post-processing steps were active; the other experimental cells (cross-technique, cross-size, cross-family, structure ablations, sampling, thinking mode) were commented out to stay within the T4’s memory limits and shorten total runtime. Under this setup, no hardware-driven adaptations were required. The commented cells can be re-enabled to reproduce the full set of experiments reported in this work.

The submission achieved a weighted F1 of 0.61346 on the Kaggle public leaderboard, consistent with the 0.60 weighted F1 computed locally on the test set (Section 8.3).

## 10.4. Future Work

Section 9.6 identified two main avenues for improvement: task-specific weight updates and richer multi-step prompting. On the fine-tuning side, parameter-efficient methods such as LoRA and QLoRA [4] add a small number of trainable adapter parameters to a frozen base model, making it feasible to fine-tune small open-weight LLMs on a single GPU. On the prompting side, scaling Generated Knowledge to the full 20 demonstrations recommended by the original formulation [5], ideally hand-crafted by domain experts, would test whether the modest setup used here is the limiting factor.

Beyond these two avenues, additional compute would enable several methodological improvements: multi-seed evaluation of the sampling and Self-Consistency experiments to characterize run-to-run variability, running Self-Consistency on the full validation set rather than a 100-sample subset, and extending the cross-technique run to larger Qwen3.5 sizes and additional model families.

## References

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.
- [2] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [3] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2026. Online manuscript released January 6, 2026.
- [4] Manolis Koubarakis. Course slides: Advanced llms, 2026. Artificial Intelligence II Course, National and Kapodistrian University of Athens.
- [5] Jiacheng Liu, Alisa Liu, Ximing Lu, Sean Welleck, Peter West, Ronan Le Bras, Yejin Choi, and Hannaneh Hajishirzi. Generated knowledge prompting for commonsense reasoning. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [6] Microsoft. Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs. *arXiv preprint arXiv:2503.01743*, 2025.
- [7] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*. 2019.
- [8] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026.
- [9] Elvis Saravia. Prompt Engineering Guide. <https://github.com/dair-ai/Prompt-Engineering-Guide>, 12 2022.

- [10] Gemma Team. Gemma 3. 2025.
- [11] Konstantinos Thomas, Giorgos Filandrianos, Maria Lymperaïou, Chrysoula Zerva, and Giorgos Stamou. “I never said that”: A dataset, taxonomy and baselines on response clarity classification. *arXiv preprint arXiv:2409.13879*, 2024.
- [12] Konstantinos Thomas, Giorgos Filandrianos, Maria Lymperaïou, Chrysoula Zerva, and Giorgos Stamou. CLARITY: SemEval 2026 task 6, 2026.
- [13] Myrto Tsokanaridou. Lab slides: Classification using prompting techniques, 2026. Artificial Intelligence II Course, National and Kapodistrian University of Athens.
- [14] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.
- [15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [16] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Barburescu, Morgan Besse, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.
- [17] Tongshuang Wu, Michael Terry, and Carrie J. Cai. AI chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, 2022.

## A. AI Assistance

In preparing this work, we used the following AI tools and disclose them here.

**Code autocompletion (Google Colab AI).** The notebook editor’s built-in AI auto-complete was used for low-level coding assistance. Generated suggestions were reviewed and edited before inclusion.

**Paper summarization (Google NotebookLM).** NotebookLM served as a quick way to read through the prompting papers cited in this report (Chain-of-Thought, Generated Knowledge, Self-Consistency, and others). All citations and technical claims were verified against the original papers.

**Report drafting (Anthropic Claude).** Claude was used as an interactive writing assistant for the report itself: refining wording, and checking consistency across cross-references. The technical content, experimental design, results, and analytical claims are our own; Claude assisted with their presentation in academic writing.